

Command Nexus (lite) — District Deployment Guide

Staff + Student management, self-hosted

Command Nexus Lite

2026

What this is

Command Nexus is a self-hosted web app for K-12 district IT operations. This “lite” distribution ships the two core modules — **Staff** and **Student Roster** — plus everything they need to work end-to-end: onboarding workflow, ID cards, guidance queue, attendance reporting, custom-section sync, provisioning, and audit logging.

Everything district-specific in the original full-featured deployment (door access, core switch ACL editing, phone system, vape sensors, camera VMS) has been stripped out. The stack is auth-flexible: **local accounts work out of the box**, and Google Workspace SSO is a feature-flagged addition for districts that use it. SIS ingestion assumes PowerSchool-style CSV exports but the column mapping is configurable so other SIS platforms work with Settings-only changes.

What you’ll be running

The stack is 5 Docker containers plus an optional 6th watchdog:

Container	Image	Purpose
nginx	nginx:1.27-alpine	Reverse proxy, TLS termination
postgres	postgres:16	Primary datastore
redis	redis:7-alpine	Session store + job queue
api	built from app/Dockerfile	FastAPI web app
worker	same image as api	ARQ background jobs
scheduler	same image as api	Cron-style job enqueueing
autoheal	willfarrell/autoheal:1.2.0	Restart unhealthy containers automatically

Total working set: ~1.5 GB RAM, ~10 GB disk after 6 months of audit logs. Any Linux VM with 4 GB RAM + 40 GB disk + Docker installed can run this comfortably.

Prerequisites you need to bring

Before you clone the repo, gather these — the first-run script will prompt for them.

Infrastructure

- **A Linux VM or bare-metal host** running Ubuntu 22.04+ / Debian 12+ / RHEL 9+ or similar. Docker Desktop on Mac/Windows works for development but production wants Linux.
- **Docker Engine 24+** and **Docker Compose v2** installed.
- **A domain name** pointing to the host, e.g. nexus.mydistrict.org.
- **A TLS plan** — three paths, pick one. See the TLS section below for setup details:
 1. **Caddy auto-TLS** (recommended if you don't already have a cert workflow): swap in the included Caddy compose override. Auto- provisions and auto-renews Let's Encrypt certs. Needs ports 80 + 443 open to the Internet.
 2. **nginx + certbot**: keep the default nginx proxy, run certbot on the host, drop certs into ./nginx/certs/.
 3. **Cloudflare Tunnel or existing reverse-proxy / load balancer**: nginx terminates HTTP from the backend; whatever's in front handles TLS.
- **Outbound Internet access** from the host to googleapis.com, google.com, PyPI, Docker Hub, and your MetaSolutions SFTP/Gmail endpoints.

Google Workspace (OPTIONAL)

Only required if you want to use Google features. Skip this entire subsection if you're deploying local-auth-only without Google backend integrations.

- **Super-admin access** to your Google Workspace tenant.
- **A project in Google Cloud Console** you can create a service account in.
- Willingness to enable **Domain-Wide Delegation** for that service account against these scopes. Paste the full list, comma-separated, into the Admin Console (Security → API controls → Domain-wide delegation → Add new → Client ID + the scope list below).

```
https://www.googleapis.com/auth/admin.directory.user,  
https://www.googleapis.com/auth/admin.directory.user.readonly,  
https://www.googleapis.com/auth/admin.directory.group,  
https://www.googleapis.com/auth/admin.directory.group.readonly,  
https://www.googleapis.com/auth/admin.directory.group.member,  
https://www.googleapis.com/auth/admin.directory.group.member.readonly
```

```
https://www.googleapis.com/auth/admin.directory.orgunit.readonly,  
https://www.googleapis.com/auth/apps.groups.settings,  
https://www.googleapis.com/auth/apps.licensing,  
https://www.googleapis.com/auth/gmail.readonly,  
https://www.googleapis.com/auth/gmail.modify,  
https://www.googleapis.com/auth/gmail.send,  
https://www.googleapis.com/auth/spreadsheets,  
https://www.googleapis.com/auth/spreadsheets.readonly,  
https://www.googleapis.com/auth/drive
```



Why each scope:

- **admin.directory.user[.readonly]** — create / suspend / move / read student + staff Google accounts.
- **admin.directory.group[.readonly / .member[.readonly]]** — manage Google Group membership for role-based access.
- **admin.directory.orgunit.readonly** — read the OU tree so Nexus can validate target OU paths before moving an account.
- **apps.groups.settings** — set group-level settings (whoCanJoin, whoCanPostMessage, etc.) on auto-created groups.
- **apps.licensing** — assign the correct license SKU (Workspace for Education, Google for Edu Plus, etc.) when creating a new student account.
- **gmail.readonly / gmail.modify** — poll the SIS mailbox for daily roster CSVs + mark them processed via label.
- **gmail.send** — send guidance queue notifications (Nexus impersonates a district admin as the From: address).
- **spreadsheets[.readonly]** — read per-building Staff Directory sheets + room roster sheets.
- **drive** — list / download the sheets referenced above by their Drive IDs.

Nexus impersonates a district admin mailbox for Gmail send/read, and hits Directory API as the service account itself.

SIS (student information system)

Nexus’s roster and attendance pipelines are built around **PowerSchool with MetaSolutions ReportingServices** as the export backend. If your district uses the same, no work needed — just point Nexus at the reports.

If you’re on a different SIS (Skyward, Aeries, Infinite Campus), the CSV import path is generic — you’ll need to map your export’s column names onto Nexus’s expected schema via Settings. See the SIS section below.

Reports Nexus consumes:

Report	Cadence	Delivery
Students (roster snapshot)	Daily	Emailed CSV to a dedicated mailbox
Daily Attendance	Daily	Emailed CSV
Membership status (A/R/F/CTC/etc.)	Daily	Emailed CSV

Local auth — the default (~5 min)

Email + password authentication, admin-managed. **Enabled by default**; set `LOCAL_AUTH_ENABLED=false` in `.env` to disable (only sensible if you’re also enabling Google SSO as the sole auth path — the API will refuse to start if both are off).

Local auth is the right default for most deployments. It works out of the box with zero external dependencies, serves as break-glass auth even in Google-primary shops, and lets Microsoft 365 districts run

Nexus without Workspace at all.

Default admin on first boot

If `LOCAL_AUTH_ENABLED=true` AND the `local_users` table is empty, the API creates a default admin on first boot:

Field	Value
Email	admin@local
Password	changeme123!
must_change_password	true

The first time anyone logs in with these creds, Nexus FORCES a password reset before letting them reach any other page. You can skip this bootstrap by running `scripts/first_run.sh` with the flag enabled — it prompts for a real email + password and seeds that instead.

Managing accounts after boot

Settings → Access → Local Accounts (admin only).

Shows every local account with email, name, admin/active flags, last login, and a “(must change pw)” marker where applicable. Per-row actions: - **Reset pw** — prompt for a new password; user is forced to change it again on next login - **Disable / Enable** — soft-disable without deleting (audit trail preserved) - **Make admin / Revoke admin** — toggle the admin bit - **Delete** — permanent

Add-account form at the bottom: email, display name, password (≥ 12 chars), admin checkbox.

Security defaults

- argon2id password hashing (OWASP-recommended)
- Per-email failure lockout: 5 attempts within 15 minutes → 15-minute lockout. Tune via `LOCAL_AUTH_LOCKOUT_FAILURES` and `LOCAL_AUTH_LOCKOUT_WINDOW_SEC` env vars if your policy differs.
- All login attempts audit-logged (success + failure)
- Session cookie identical to the Google flow — downstream middleware can’t distinguish the two auth paths
- Admin resets automatically set `must_change_password=true` unless the admin passes `skip_force_change=true` on the PATCH

Google service account setup — OPTIONAL (~15 min, only if you want Google backend features)

1. Create the Google Cloud project

- Open <https://console.cloud.google.com/>.

- Click the project dropdown at the top, then **New Project**. Name it something like <district>-nexus.
- With the new project active, go to **APIs & Services → Library** and enable:
 - Admin SDK API
 - Gmail API
 - Google Sheets API
 - Google Drive API

2. Create the service account

- In the same project, go to **IAM & Admin → Service Accounts → Create Service Account**.
- Name it nexus-runtime; you can skip granting Project roles (Nexus doesn't use GCP for compute, only DWD).
- Click into the newly-created account, then **Keys → Add Key → Create new key → JSON**. Save the file somewhere you can open it in a text editor later — Nexus has two ways to consume it:
 - **Recommended:** paste the full JSON into Settings → General → Google Workspace → *Service Account JSON key* once you're logged in. Stored encrypted in the database. No shell access needed to the host.
 - **Alternative** (classic docker-secrets flow): copy the file to the Nexus host as secrets/google_service_account.json before first boot. Both paths work; the Settings paste wins if both exist.

3. Enable Domain-Wide Delegation

- Copy the service account's numeric **Client ID** (starts with 1078... or similar).
- Open <https://admin.google.com>, then go to **Security → Access and data control → API controls → Manage Domain Wide Delegation**.
- Click **Add new**, paste the Client ID, and paste the scopes list from the Prerequisites section above (all on one line, comma-separated).

4. Grant admin roles

Nexus never needs full super-admin. Create a **custom admin role** in admin.google.com → Account → Admin roles → Create new role with these privileges, then assign it to the admin mailbox Nexus will impersonate (often a dedicated nexus.svc@yourdomain.org):

- Organizational Units (read/write)
- Users (read/write)
- Groups (read/write)
- Reports (read)

**Google OAuth 2.0 client —
OPTIONAL (~5 min, only if
GOOGLE_AUTH_ENABLED=true)**

The **service account** above handles Nexus's backend API calls (Directory, Gmail, Sheets). **User login** uses a separate OAuth 2.0 Client ID so staff can sign in with their district Google account via the "Sign in with Google" button on the login page.

Both live in the same Google Cloud project you created above.

1. Create the OAuth consent screen

- In your project, go to **APIs & Services → OAuth consent screen**.
- User Type: **Internal** (restricts to users in your Workspace domain — exactly what you want for staff login).
- Fill in:
 - **App name**: Command Nexus (or your district's name for it)
 - **User support email**: an admin mailbox
 - **Developer contact**: same
- Scopes: add openid, email, profile. Those are the only three the login flow needs.
- Save and continue through the rest; no test users to add for an Internal app.

2. Create the OAuth 2.0 Client ID

- **APIs & Services → Credentials → Create Credentials → OAuth client ID**.
- **Application type**: Web application.
- **Name**: Nexus Web Login (anything, it's just a label).
- **Authorized redirect URIs** — add one or more:

`https://nexus.yourdistrict.org/auth/callback`

(Match your real DOMAIN from `.env`. The path `/auth/callback` is fixed — Nexus's OAuth router expects it there.)

For internal / LAN-only deployments (TLS Paths D + E): register the LAN hostname instead:

`https://nexus.district.local/auth/callback`

Google doesn't validate reachability during registration — a non-public hostname works fine as long as the URI matches EXACTLY what Nexus sends in the OAuth redirect. The browser doing the login needs to reach that hostname (so staff must be on the LAN or VPN).

For mixed internal/external access (same deployment reachable via both a LAN name and a public name): add BOTH redirect URIs to the same OAuth Client — Google allows multiple. Nexus always sends the redirect URI matching its current DOMAIN env var.

- Click Create.
- Copy the **Client ID** (ends in `.apps.googleusercontent.com`) and **Client secret** (shown once; you can regenerate if you miss it).

3. Drop them into secrets

scripts/first_run.sh prompts for both and writes them to secrets/google_client_id and secrets/google_client_secret. Or place them manually before running the script:

```
echo -n '<your-client-id>.apps.googleusercontent.com' >
secrets/google_client_id
echo -n '<your-client-secret>' >
secrets/google_client_secret
chmod 600 secrets/google_client_*
```

That's it — no DWD, no scopes list, no admin console changes needed for the OAuth client. The service account's DWD is a totally separate authorization that governs backend API access.

TLS setup — pick ONE path

Everything after this assumes you have a DOMAIN set in .env that resolves to this host. Pick the TLS strategy that fits your environment and skip the other two.

Path A — Caddy auto-TLS (easiest, recommended)

Caddy auto-provisions and auto-renews Let's Encrypt certificates with no cron, no config, no manual intervention. Zero certificate maintenance for the life of the deployment.

Requirements: ports **80 + 443** open from the public Internet to this host, DNS A/AAAA record for your DOMAIN pointing at this host.

```
# Confirm CADDY_ACME_EMAIL is set in your .env (expiry alerts from
# Let's Encrypt go there), then bring the stack up with the Caddy
# override instead of the default nginx:
docker compose -f docker-compose.yml -f docker-compose.caddy.yml up
-d
```

Caddy prints the Let's Encrypt handshake in its log the first time:

```
docker compose -f docker-compose.yml -f docker-compose.caddy.yml
logs caddy
```

Within ~30 seconds you'll see certificate obtained successfully and https://your-domain/ works.

Path B — nginx + certbot

Keep the default nginx proxy. Run certbot on the host and mount the resulting cert directory into the nginx container.

```
# One-time cert issue
sudo apt install certbot # or dnf
install certbot
sudo certbot certonly --standalone -d nexus.yourdistrict.org \
--email admin@yourdistrict.org --agree-tos --non-interactive
```

```

# Point nginx at the cert – edit nginx/conf.d/default.conf, replace
# the "listen 80" block with:
#   server {
#       listen 443 ssl http2;
#       ssl_certificate
/etc/letsencrypt/live/<domain>/fullchain.pem;
#       ssl_certificate_key
/etc/letsencrypt/live/<domain>/privkey.pem;
#       ... (keep the location / proxy_pass block as-is)
#   }

# Mount /etc/letsencrypt into the nginx container by adding to the
# nginx service in docker-compose.yml:
#   volumes:
#       - /etc/letsencrypt:/etc/letsencrypt:ro

docker compose up -d

```

Add a weekly cron for renewal:

```

echo '0 3 * * 0 root certbot renew --quiet && docker exec nexus-
lite-nginx nginx -s reload' \
| sudo tee /etc/cron.d/nexus-lite-cert-renew

```

Path C — Cloudflare Tunnel or external TLS termination

If you already run Cloudflare Tunnel, F5, HAProxy, or another edge appliance that terminates TLS before traffic reaches this host:

- Leave the stock nginx config as-is (HTTP-only on 8080)
- Point your tunnel/LB at `http://<this-host>:8080`
- Make sure your edge sets `X-Forwarded-Proto: https` so Nexus knows to generate HTTPS URLs in redirects and emails

No Docker changes needed for this path.

Path D — Local-only (LAN deployment, no Internet exposure)

For districts that want to run Nexus entirely on the internal network with no public DNS and no inbound Internet reachability. The included `docker-compose.local.yml` + `Caddyfile.local` use Caddy's internal CA to self-sign a cert for your LAN hostname.

```

# Set DOMAIN in .env to your LAN hostname, e.g. nexus.district.local
docker compose -f docker-compose.yml -f docker-compose.local.yml up
-d

```

One-time per device: users will see a browser “Not secure” warning until they trust Caddy's root CA. Fetch and distribute it with:

```

# Fetch the root cert from the running Caddy container
docker cp nexus-lite-
caddy:/data/caddy/pki/authorities/local/root.crt \
nexus-lite-root.crt
# Push via GPO (Windows), MDM (ChromeOS/iPadOS), or install manually

```

Google OAuth still works because the callback URL is HTTPS even with a self-signed cert; Google doesn't verify the cert, only the scheme. Users will see the browser warning once per device until the root CA is trusted, but OAuth itself has zero issues.

Path E — DNS-01 (real Let's Encrypt cert, no inbound 80/443 needed)

Best-of-both-worlds for LAN deployments that want a real public-CA cert but can't open 80/443 to the Internet. Uses Caddy's DNS-01 ACME challenge: Caddy proves domain ownership by writing a TXT record to your public DNS zone (via your DNS provider's API), gets a Let's Encrypt cert, no inbound ports needed.

Requirements: - A public DNS zone for your DOMAIN (e.g. nexus.yourdistrict.org), with API access on the registrar/DNS host - API token with permission to write TXT records on that zone - Split-horizon DNS: public DNS can point anywhere (even a bogus IP); internal DNS resolves the hostname to this host so staff reach it

Default plugin is **Cloudflare** (most common free-tier DNS). To use a different provider edit docker/caddy-dns01/Dockerfile (DNS_PLUGIN build arg) + the matching acme_dns line in Caddyfile.dns01. The full list of supported providers: <https://github.com/caddy-dns>

```
# 1. Add to .env:
#     CADDY_ACME_EMAIL=admin@yourdistrict.org
#     CLOUDFLARE_API_TOKEN=<token with Zone:DNS:Edit scope>
# 2. Build + launch
docker compose -f docker-compose.yml -f docker-compose.dns01.yml
build caddy
docker compose -f docker-compose.yml -f docker-compose.dns01.yml up
-d
```

Caddy prints certificate obtained successfully within ~60 seconds (DNS propagation + ACME round-trip). Zero browser warnings for staff since the cert is from a public CA.

Installing on the host (~15 min)

Prereq installer (one time, as root)

If this is a fresh host, the included scripts/install_prereqs.sh installs Docker Engine (official repo, not the distro default which is often ancient), the Docker Compose plugin, git, and curl. Works on Ubuntu, Debian, RHEL, Rocky, AlmaLinux, CentOS, and Fedora.

```
# As your regular (non-root) deploy user, in a scratch dir:
curl -fsSL https://raw.githubusercontent.com/timwellington-
alt/command-nexus.lite/master/scripts/install_prereqs.sh \
-o install_prereqs.sh
chmod +x install_prereqs.sh
sudo ./install_prereqs.sh
# Log out + back in so your docker group membership takes effect.
```

The script is idempotent — safe to re-run if it bails partway. Last step adds you to the docker group; you MUST log out + back in before docker commands work without sudo.

Clone + configure

```
# 1. Clone
git clone https://github.com/timwellington-alt/command-
nexus.lite.git \
    command-nexus-lite
cd command-nexus-lite

# 2. Copy the env template
cp .env.example .env
$EDITOR .env      # fill in DISTRICT_NAME, DOMAIN, TIMEZONE

# 3. Drop your Google service account key
cp ~/nexus-runtime-key.json secrets/google_service_account.json
chmod 600 secrets/google_service_account.json

# 4. Run the first-run script – generates DB/redis passwords,
#     session keys, and prompts for the initial admin email.
./scripts/first_run.sh

# 5. Bring up the stack
docker compose up -d

# 6. Watch startup
docker compose logs -f api scheduler worker
```

After ~30 seconds, the stack should be healthy:

```
docker compose ps
```

All containers should read Up (healthy). If any read unhealthy after 2 minutes, tail its logs:

```
docker compose logs --tail 50 <container-name>
```

Common first-boot fixes:

- **Migrations failed → check DB permissions.** The `postgres_password` file must match the actual password Postgres was initialized with; a mismatch after a re-run means you need to `docker compose down -v` to reset the volume.
- **Google API errors → DWD scopes missing.** Re-check the scope list in the Google Admin console.

First login (~5 min)

- Navigate to `https://<your-domain>/`.
- Log in with the admin email you seeded via `first_run.sh`.
- You'll land on `/dashboard`. Most tiles will be dim until you fill out settings — that's expected.

Configuration walkthrough (~30

min)

Go to /settings. Fill these panels in order:

Branding

- **District name** — used in the page header and outbound emails.
- **Logo URL** — small transparent PNG works best (128×128 or 256×256).
- **Primary color** — for accents. Defaults to a neutral blue.

Buildings

- Add each of your schools as a row: Internal code | Display name | SIS code.
- The **Internal code** is what Nexus uses in nav/URLs (short, all-caps, e.g. PES, PHS).
- The **SIS code** is whatever your SIS report emits (e.g. SIS_A, SIS_B) — Nexus keeps a translation table so users never see raw SIS codes.

Google

- **Domain** — your primary Workspace domain (e.g. mydistrict.org).
- **Admin impersonation email** — the mailbox the service account impersonates.
- **Student OU map** — per-building, the OU path new student accounts land in (e.g. PES → /Students/PES).
- **Staff OU map** — same, for staff (e.g. PES → /Staff/PES).

SIS ingest

- **Report sender email** — the address MetaSolutions (or your SIS exporter) sends reports FROM.
- **Roster subject filter** — Gmail search string, e.g. subject:"Students".
- **Attendance subject filter** — e.g. subject:"Daily Attendance".
- **Membership subject filter** — e.g. subject:"Student Membership executed at".

The **Roster ingest** page has a per-column mapping panel where you name each column of your students.csv export against Nexus's expected fields (sis_id, first_name, last_name, grade, school, email, etc.). Once mapped, the next poll picks it up automatically.

HR classification map

- Your HR system uses codes (Cert, Class, Adm, Exempt, etc.) that Nexus maps to role_type (teacher, admin, sub, tech, aide, other).
- Fill this in the HR panel or accept the defaults.

Attendance reports

- **Recipients** — the emails that receive the daily digest, grouped by building.
- **Send time** — hour + minute in district TZ (defaults to 11:00).
- **Weekdays** — Mon-Fri by default.

Guidance queue routing

- **Counselor groups** — building + grade range → counselor email (e.g. PHS grades 9-10 → aparker@...). Nexus uses this to attribute enrollment / withdrawal / transfer queue entries to the right person.

Verification checklist

Before handing off to daily operators, confirm these end-to-end:

- ☐ A user in your admin role logs in and lands on /dashboard.
- ☐ A user in a lower role logs in and DOES NOT see admin-only tiles.
- ☐ /roster renders your students (may be empty on day 1 — poll hasn't run yet).
- ☐ /staff renders your staff directory.
- ☐ Trigger a manual sync from **Settings** → **Google** → **Sync now** and confirm rows populate.
- ☐ Trigger /api/roster/attendance-reports/preview (or the “Preview” button in the Attendance panel) and confirm a sample email renders.
- ☐ docker compose ps shows all containers healthy.
- ☐ /settings **Job Health** panel shows every scheduled job in green after the first tick.

Ongoing operations

Daily

Nothing manual — the scheduler runs everything.

Weekly

- Skim **Settings** → **Job Health**: any red row means a job hasn't succeeded in >3× its cadence. Click into it for the last error.
- Skim **Settings** → **Audit** for anything unexpected.

Monthly

- docker compose pull + docker compose up -d to update to the latest release, then confirm all healthy.
- Rotate the Google service account key if your security policy requires it — drop the new JSON in place, restart api + worker.

Backups

- postgres is the only stateful container. Its data lives in the named volume pg-data.
- Nightly pg_dump recommended; example script in scripts/nightly_backup.sh. Encrypt the dump and ship it off-host.

Troubleshooting

“Container X is unhealthy”

- API: check `docker compose logs api` — usually a DB connection or Google auth error at startup.
- Worker: `docker compose logs worker` — an FD leak or DB drop can wedge it. Autoheal will restart within 60s but the underlying issue is worth reading.

“No absence records showing today”

- Check that today’s Daily Attendance email actually arrived at your Nexus mailbox.
- Manually enqueue: `docker compose exec worker python -c "import asyncio; from app.workers.clever_import_job import poll_clever_imports; print(asyncio.run(poll_clever_imports({})))"`

“Google API 403 permission_denied”

- 99% of the time this is missing or wrong DWD scopes. Recheck the scope list in Google Admin against this doc.

“Staff profile shows wrong photo / wrong data”

- Trigger `run_staff_reconciliation` manually from **Settings → Job Health → Run now**. The reconciler is the arbiter of truth; other syncs feed it.

Getting help

- Issues + questions: open a GitHub issue on the template repo.
- Time-critical outages: this is self-hosted; there is no vendor support line. Roll back with `docker compose down + revert to the last known-good tag with git checkout <tag> + docker compose up -d`.